

LogiFlow: DevSecOps Threat Monitoring & Hardening



*Andersson David Sánchez Méndez
Cristian Santiago Pedraza Rodríguez
Jeisson David Sánchez Gómez*

Date: May 02, 2026

Repo / Demo:

- <https://github.com/AnderssonProgramming/devsecops-threat-monitor>
- <https://github.com/AnderssonProgramming/logiflow-cybersecurity-seminar>



Agenda



Problem
statement & goals



System overview
(LogiFlow)



OWASP attack
surface



STRIDE + Risk
Matrix



Architecture
(Before/After)



DevSecOps
Pipeline



SAST/DAST Results



Runtime
Monitoring



Test Plan



Traceability



Problem Statement

- **Problem:** LogiFlow was initially deployed without proper security controls in production.

- **Critical risks identified:**



No rate limiting on webhook endpoints



No authentication in WebSocket connections



Verbose stack traces exposed in API

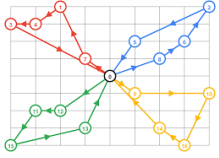


GPS coordinates stored in plain text

- **Goal:**



Demonstrate continuous security monitoring and system hardening with real, testable evidence.



System Overview (LogiFlow)

- **Purpose:** Real-time fleet routing platform that solves the Vehicle Routing Problem (VRP) — calculates and recalculates optimal routes for N vehicles and M delivery points in less than 4 seconds.
- **Tech stack:**
 - Gateway/API: NestJS · GraphQL · REST · JWT
 - Optimizer: VROOM (C++) via gRPC — VRP solver
 - Realtime: Socket.io + Redis Adapter
 - Automation: n8n · external event webhooks
 - Frontend: Angular + Ionic (driver + dispatcher)
- **High-level architecture:**
 - n8n detects external event → NestJS Gateway orchestrates → gRPC to VROOM → optimal routes → Socket.io pushes to real-time drivers





OWASP Attack Surface

A01

HIGH

Broken Access Control

Socket.io rooms without JWT

A02

HIGH

Cryptographic Failures

Redis :6379 unencrypted

A03

HIGH

Injection

n8n webhooks without validation

A05

HIGH

Security Misconfiguration

NestJS dev mode (stack traces)

A06

MEDIUM

Vulnerable Components

npm without security audit

A07

HIGH

Auth Failures

Expired JWT accepted in WS

A09

MEDIUM

Logging Failures

No alerts or log correlation

A10

MEDIUM

SSRF/Insecure Design

Full trust between services



STRIDE + Risk Matrix

Threat	Risk	Attack Surface	Control
Spoofing	HIGH	Stolen JWT → impersonates driver in WS	JWT validation + Falco
Tampering	HIGH	gRPC payload without TLS	Inter-service TLS + validation
Repudiation	MEDIUM	No audit log in gateway	Loki + audit trail
Info Disclosure	HIGH	Stack traces + GPS in logs	Log redaction + Redis AUTH
Denial of Service	HIGH	n8n webhook without rate limit	Rate limiter + Prometheus
Elevation	MEDIUM	Driver/admin roles undifferentiated	RBAC + Falco rules

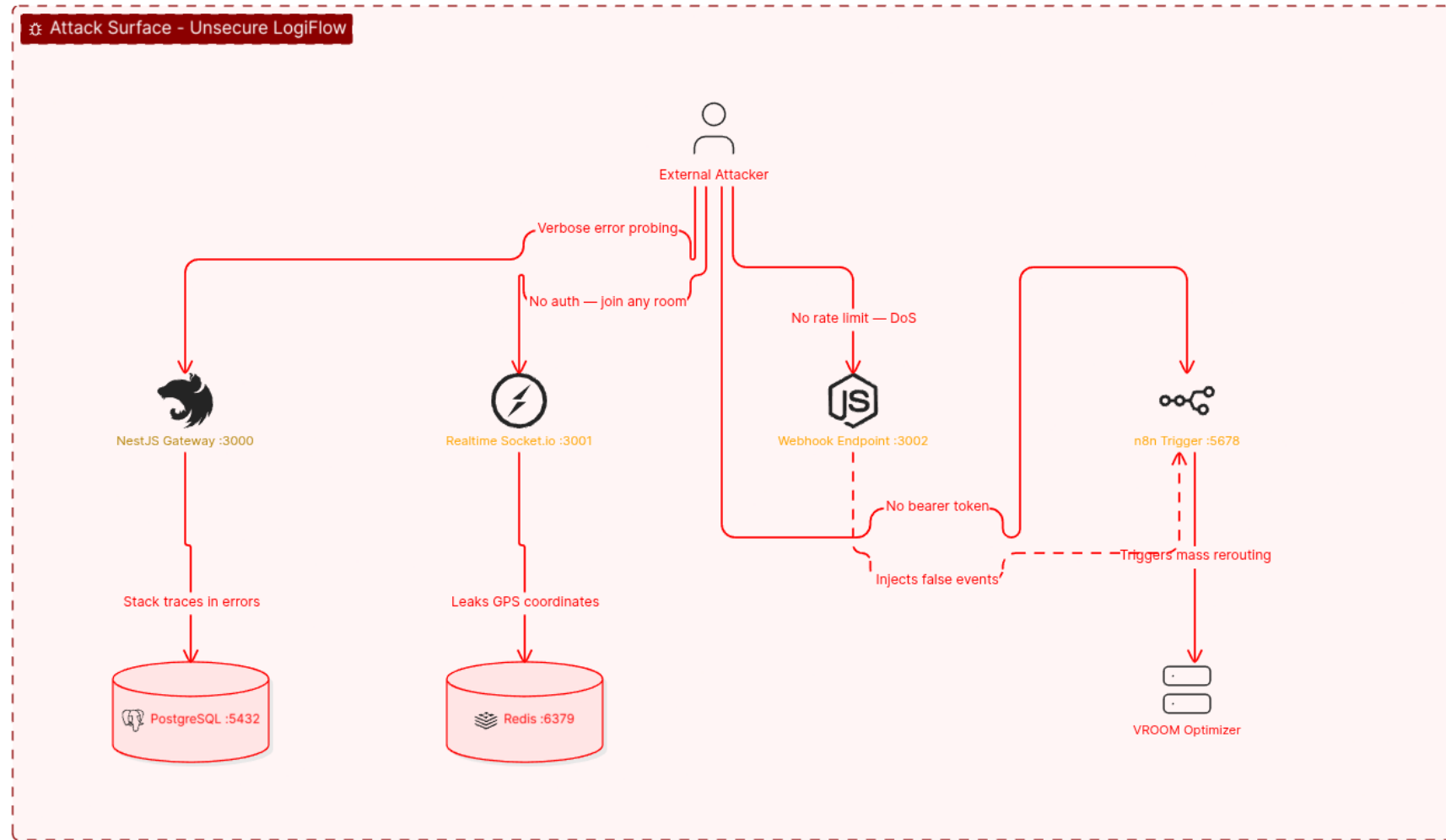


Insecure Architecture (Before)

- **Trust boundary issues:**
 - Flat network between services
 - Minimal authentication between internal components
- **Common weaknesses (callouts):**
 - Secrets accessible in CI logs / env
 - Over-permissive service accounts/tokens
 - Publicly reachable admin/metrics endpoints
 - Limited validation on ingestion pipeline
- **Attack vectors to annotate on diagram:**
 - External attacker → API abuse → data exfiltration
 - Compromised pipeline token → deploy malicious build
 - Ingestion flood → resource exhaustion (DoS)



Insecure Architecture Diagram (Before)





Secure Architecture (After)



Auth + RBAC

JWT required on WebSocket.
Driver/admin roles differentiated.



Network Segmentation

Allow-lists between internal
services. No implicit trust.



Secrets Management

No secrets in repository. Minimum-
scope tokens.



Input Validation

JSON schema in n8n + DTO
validation in NestJS.



TLS + Secure Headers

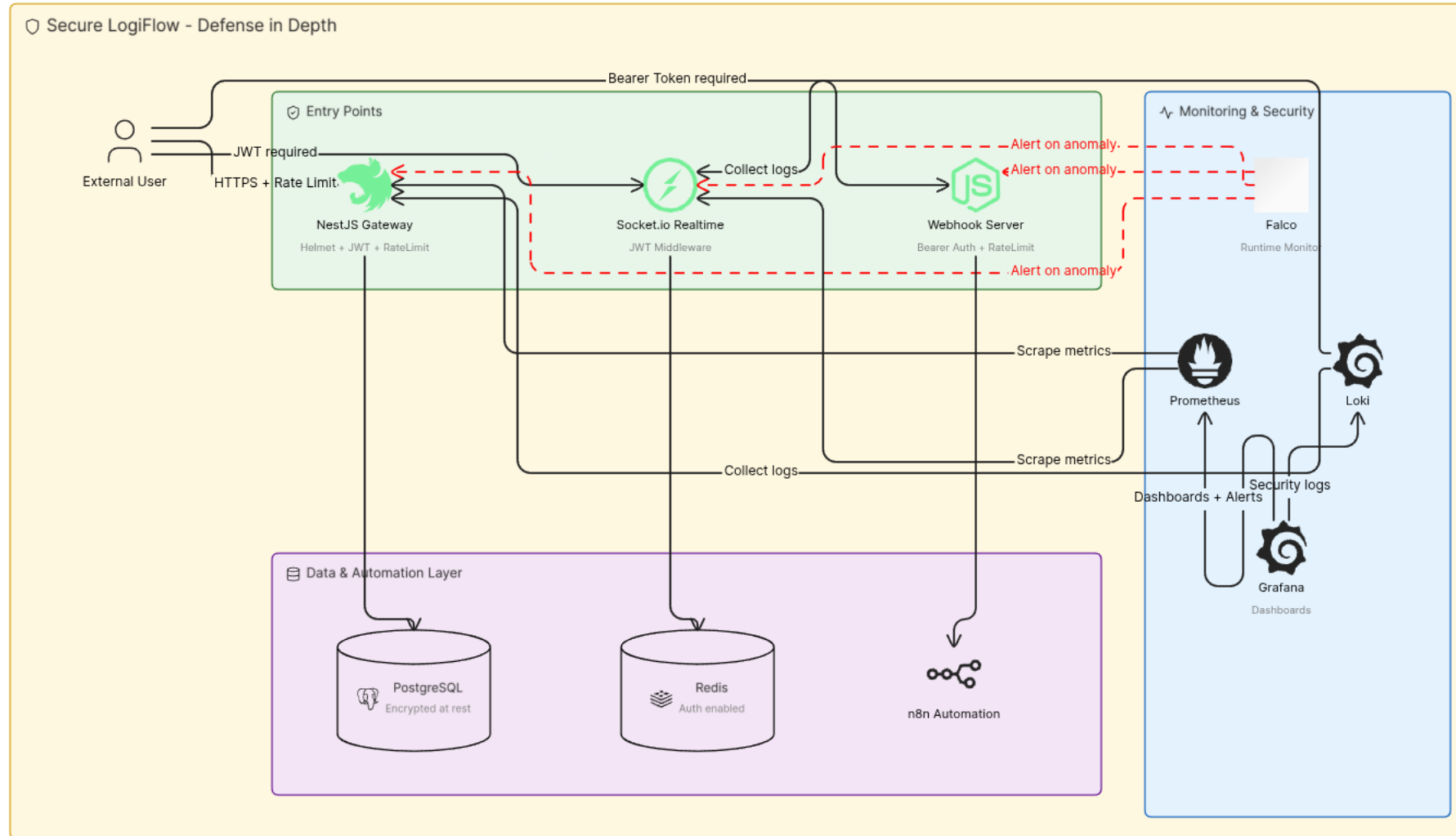
Encrypted inter-service transport.
Helmet in NestJS.



Runtime Monitoring

Falco + alerts. Centralized audit log
with Loki.

Secure Architecture Diagram (After)





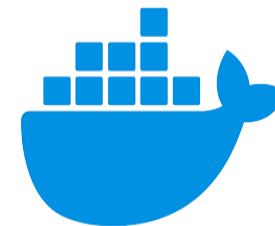
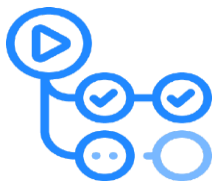
DevSecOps Pipeline



DEV = Development Stages

SEC = Security Analysis

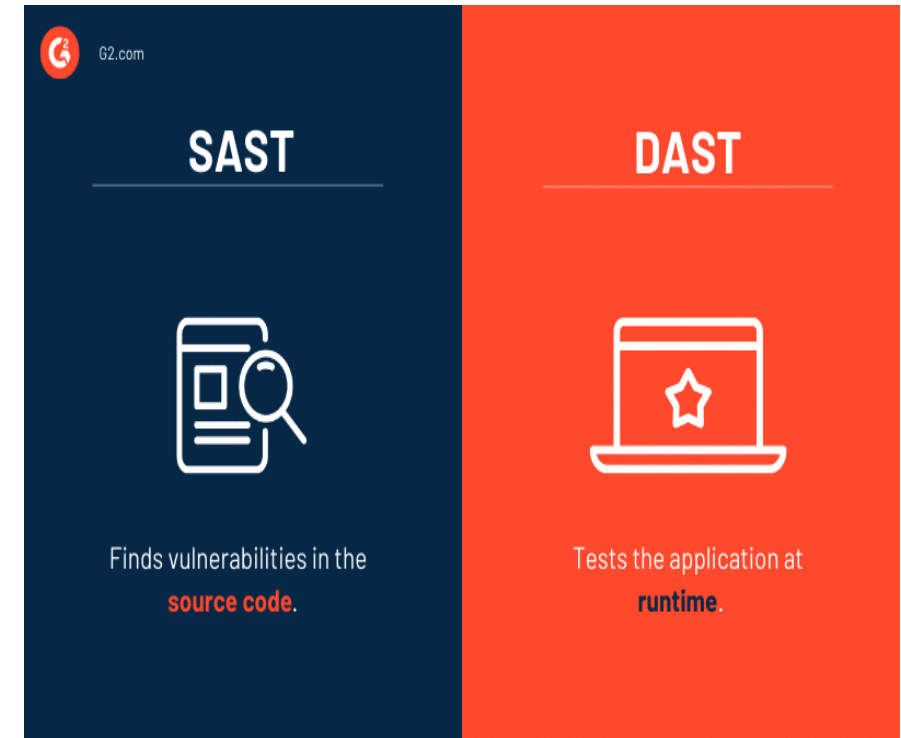
OPS = Deployment & Monitoring





SAST / DAST Results (CodeQL + Semgrep)

- **What we scanned:** application code + configs
- **Tools:** GitHub CodeQL, Semgrep ruleset (security-focused)
- **Findings (example categories):**
 - Unsafe input handling / missing validation
 - Hardcoded secrets patterns / weak secret hygiene
 - Insecure logging (sensitive info leakage)
 - Dependency flags (where applicable)
- **Remediation examples:**
 - Added validation/sanitization at API boundaries
 - Removed sensitive logging; added redaction
 - Tightened auth checks; centralized authorization helper
- **Result:** fewer high/critical findings after fixes



Runtime Monitoring (Falco)

- **Why runtime monitoring:** catches what scanning misses (misuse, abuse, compromise)

- **Falco architecture:**

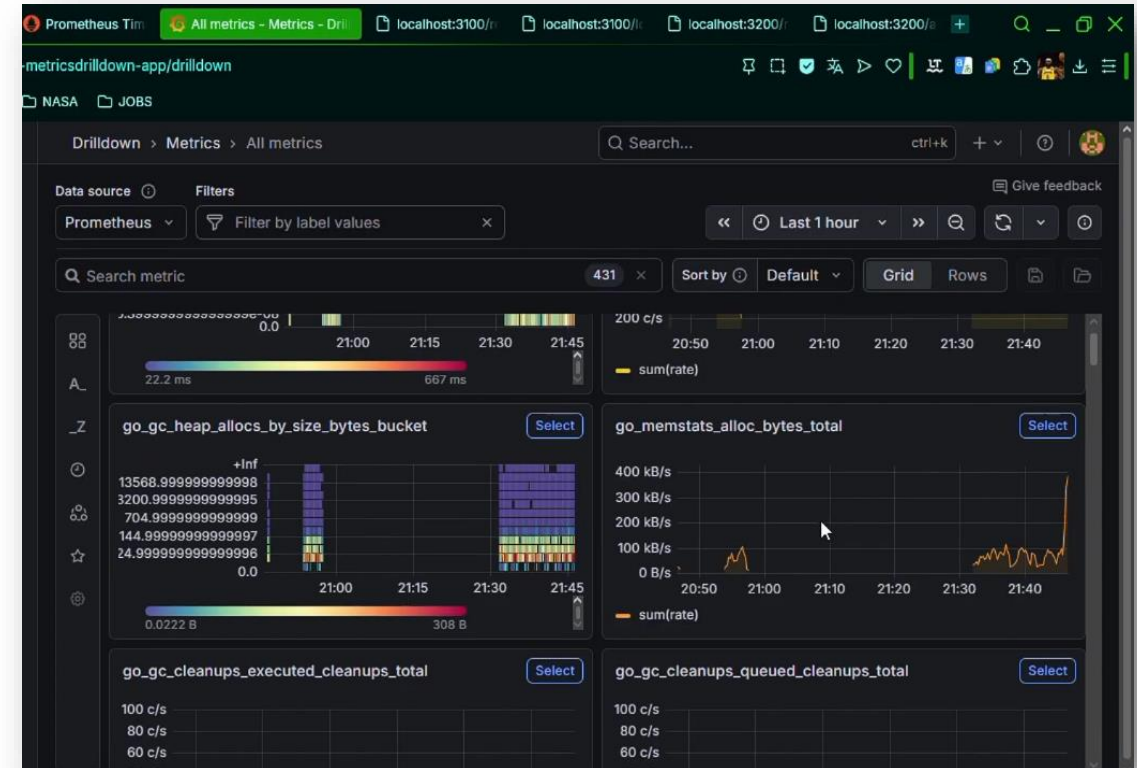
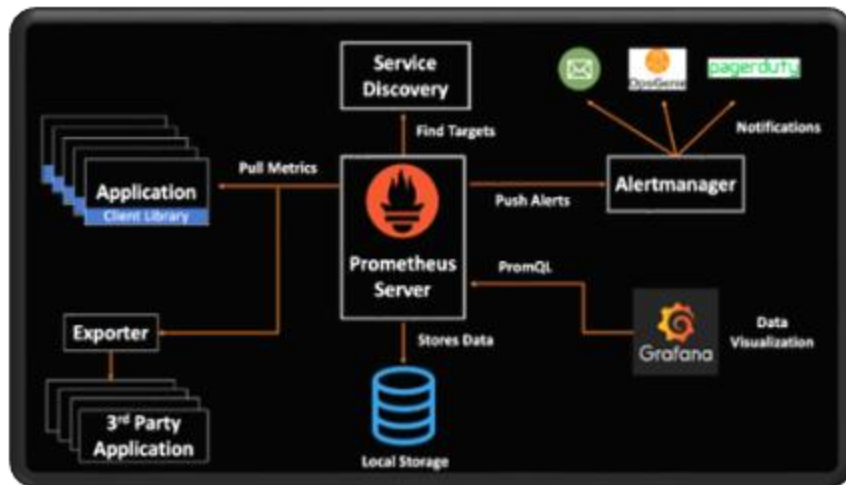


- **Custom rules (examples):**
 - Suspicious shell in container
 - Writes to sensitive paths (e.g., /etc, app secrets directory)
 - Unexpected outbound connections
 - Privilege escalation signals
- **Sample alert:** show rule name, container, process, command, severity



Grafana Dashboard (Threat Overview)

- **Dashboard panels (suggested):**
 - Alert volume by severity over time
 - Top triggered Falco rules
 - Auth anomalies (failed logins, token errors, unusual IPs)
 - Pipeline status (last run, pass/fail, scan summaries)
- **How it's used:** triage → drill-down → evidence for incidents
- **Value:** shared visibility for dev + security + ops





Test Plan Summary (TC-01 to TC-10)

- **Approach:** functional + security verification tests
- **Matrix:**
 - TC-01 ... TC-10: Pass/Fail + evidence link/screenshot
- **What tests validate:**
 - AuthZ boundaries (role-based access)
 - Input validation and safe error handling
 - Pipeline gates (build fails on high findings)
 - DAST regressions reduced
 - Runtime alerts fire on simulated suspicious behavior
- **Key evidence:** CI artifacts + screenshots + logs





SPTI Course Concepts Mapping

Course Concept	Implementation in LogiFlow	Artifact / Evidence
Threat Modeling (STRIDE)	STRIDE table + 3×3 Risk Matrix	Slide 6 · threat-model.md
OWASP Top 10	8 vulnerability mapping + mitigations	Slide 5 · owasp-mapping.md
Secure Architecture	Before/after diagram + trust boundaries	Slides 7-10 · architecture.drawio
Secure SDLC / DevSecOps	CI gates with integrated SAST/DAST	Slide 12 · .github/workflows/
Monitoring & Incident Response	Falco + Grafana + defined alerts	Slides 13-14 · falco-rules.yaml
Verification & Validation	10 test cases with evidence	Slide 15 · test-plan.md



Conclusions + Demo QR

- **Key outcomes:**
 - Identified key threats and attack paths
 - Hardened architecture with layered controls
 - Automated SAST + DAST + runtime monitoring in pipeline
 - Produced evidence: reports, dashboards, alerts, test matrix
- **Future work:**
 - Expand rule coverage + reduce false positives
 - Add alert routing (Slack/Teams/PagerDuty) + runbooks
 - More negative tests + continuous fuzzing (optional)
- **Demo:** QR code to repo + short live demo plan (5 minutes)



DevSecOps Threat Monitor
GitHub repository



LogiFlow Cybersecurity
Seminar GitHub repository



Short Live Demo Plan
Youtube Video

Thanks